

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report

Topic 4 – Async Research Assistant

Team: M501 Group 2
Date: September 19, 2026
Repo: <https://github.com/Eldar2107/research-assistant>
Members: Ulvi Ismayilzada, Eldar Samadov, Rashad Atayi

Executive Summary

We built a research assistant that takes a plain-language question, queries Wikipedia, arXiv and web search *at the same time*, and returns one short answer with numbered [N] citations. The concurrent path (`gather_sources`) costs roughly the slowest single source, while the sequential baseline (`fetch_sources_sequential`) costs roughly the sum of all three; both implementations live side by side in `src/concurrency/orchestrator.py` so the comparison is apples to apples. The measured performance metrics confirm a significant reduction in latency via asynchronous execution (§3.2).

Around that core we built a full delivery surface rather than a CLI alone: a FastAPI service with a built web page, a CLI, a PostgreSQL history store with a SQLite fallback, a file cache with TTL, and a Docker Compose stack that includes an *offline demo profile* which runs with no keys and no network. The provided `ai/` package is used exactly as distributed; only `services/AIService` crosses that boundary, which is what makes the LLM provider a one-line change in `.env`.

Contents

Executive Summary	1
1 Project Overview	3
1.1 Problem framing & scope	3
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	4
4 Robustness & Error Handling	4
4.1 Wrapping the AI module	4
4.2 Degradation paths	5
4.3 Input validation	5

5	Storage & Caching	5
5.1	History store	5
5.2	Caching policy	5
6	Testing	5
7	Deployment & Reproducibility	6
7.1	Docker	6
7.2	API surface	6
7.3	Configuration	6
8	Limitations & Future Work	6
9	Tools & Acknowledgements	7
	References	7
A	Appendix — Reproducing the project	7

1 Project Overview

1.1 Problem framing & scope

A user asks a research question in plain language, for example “*What is photosynthesis and what are its main stages?*” The system queries three public sources concurrently — Wikipedia for encyclopedic summaries, arXiv for papers, and a web search provider for everything else — and then asks a language model to write one short answer that cites the retrieved excerpts by number. The input is a single string; the output is the answer text with [N] markers, rendered in the terminal, as JSON from the API, or in the web page.

Scope is deliberately wider than the minimum brief in one direction and narrower in another. Wider: we shipped an HTTP API and a web front end on top of the CLI, plus a persistent query history in PostgreSQL. Narrower: we did not build provider failover, authentication or rate limiting, and the reference list (title and link) is not yet returned alongside the answer — see §8.

1.2 Provider choice and the AI module contract

The synthesis backend is selected by `LLM_PROVIDER` and can be `anthropic`, `openai` or `gemini`; the default is `gemini` with `gemini-2.5-flash`, because the model id lives in `.env` and can change without touching code. Web search is selected by `WEB_SEARCH_PROVIDER` and can be `tavily` (default), `serper` or `duckduckgo`; DuckDuckGo needs no key, which is what keeps a keyless run possible. Wikipedia and arXiv need no keys at all.

The provided `ai/` package comes from the course and is used as is — we did not modify its public interface. Only `services/AIService` calls `ai/synthesize`, so swapping one LLM vendor for another touches exactly one module plus the environment file.

2 Architecture

2.1 Module map

```
.
|-- ai/
| |-- providers/
| |-- sources.py
| |-- synthesizer.py
| '-- schemas.py
|-- src/
| |-- config.py
| |-- cli.py
| |-- api.py
| |-- concurrency/
| |-- services/
| '-- storage/
|-- frontend/
|-- scripts/
|-- tests/
|-- data/research_questions.json
|-- artefacts/
|-- docker/
|-- Dockerfile
|-- docker-compose.yml
|-- requirements.txt
|-- .env.example
'-- README.md
```

Listing 1: Repository layout

Module by module: `src/config.py` reads the environment into a settings object shared by every module; `src/cli.py` parses arguments and renders answers; `src/api.py` exposes the HTTP endpoints and serves the built front end; `src/concurrency/` holds both the sequential and the concurrent orchestrators; `src/services/` holds `AIService` and `CacheManager`; `src/storage/` holds the repository backed by PostgreSQL or SQLite; `frontend/` is the web page; `scripts/` holds the benchmark harness; `data/research_questions.json` holds the benchmark questions and `artefacts/` receives the reports.

2.2 Data flow across module boundaries

```
CLI / API -> concurrency/gather_sources --> Wikipedia
                                     +-> arXiv
                                     +-> Web search
      -> services/AIService --> services/CacheManager (JSON files, TTL)
                                     +-> ai/synthesize (LLM, retry + timeout)
CLI -> storage/repository (PostgreSQL or SQLite history)
```

Listing 2: How it fits together

Two boundaries carry the design. The first is `AIService`, the single caller of `synthesize`; the second is the storage repository, which hides whether history lives in PostgreSQL or SQLite behind one interface. Degradation is defined at both: if a source fails or times out it is skipped and the answer is built from the remaining sources, and if the LLM call fails while an older cached answer exists, that answer is returned with an offline note rather than an error.

3 Concurrency & Performance

3.1 Why this concurrency model

The workload is three independent network calls per question, so the bottleneck is waiting on providers — exactly what async I/O parallelizes best. Threads would add complexity for no gain against HTTP waits and processes would be absurd. `gather_sources` issues the three fetches together with a per-source timeout (`PER_SOURCE_TIMEOUT_SECONDS`, default 10 s), so one slow provider bounds its own cost instead of the whole request.

3.2 Sequential-vs-concurrent benchmark

The comparison runs `fetch_sources_sequential` (about the sum of the three sources) against `gather_sources` (about the slowest one), both from `src/concurrency/orchestrator.py`, over the questions in `data/research_questions.json`; reports are written to `artefacts/`.

Workload	N	Sequential (s)	Concurrent (s)	Speedup
5 questions × 3 sources	15 fetches	4.52	2.73	1.66×

Measurement environment: Windows, Python 3.13.3, cleared cache (–offline mode).

4 Robustness & Error Handling

4.1 Wrapping the AI module

Every call into the provided package passes through `AIService`, which adds a retry with timeout around synthesis and a per-source timeout around each fetch. Nothing else in the codebase imports `synthesize` directly, so the retry, timeout and logging policy cannot be bypassed by accident.

4.2 Degradation paths

Three failure paths are handled explicitly rather than allowed to crash a request:

- **A source fails or times out.** It is dropped and the answer is synthesized from whatever returned.
- **The LLM call fails.** If an older cached answer exists for the question, it is returned with an offline note.
- **A repeated question.** It is served from the database history with a [CACHE] prefix, which makes cache behavior visible to the user rather than silent.

4.3 Input validation

The CLI accepts a question string plus `-sources` (any subset of `wiki`, `arxiv`, `web`) and `-no-cache`; unknown source names are rejected before any network call. The API mirrors the same choice through the `sources` field on `/ask`.

5 Storage & Caching

5.1 History store

Query history is persisted through `src/storage/repository`, against the PostgreSQL container in Docker or against SQLite locally. Running outside Docker therefore needs `DATABASE_URL=sqlite:///./research_ask` in `.env`, since the default value points at the `research-postgres` host that only exists on the Compose network. Tables are created with `create_all`.

5.2 Caching policy

There are two caches. `services/CacheManager` stores synthesized answers as JSON files under `CACHE_DIR` (default `./.cache`) with a TTL from `CACHE_TTL_SECONDS` (default 24 h). Separately, the history table serves repeated questions with a [CACHE] prefix. They do not currently share keys, which is recorded as a limitation rather than presented as a design.

6 Testing

```
docker compose run --rm -e PYTHONPATH=/app app pytest --cov=src --cov-report=term-missing
docker compose run --rm -e PYTHONPATH=/app app pytest tests/test_ai_smoke.py -v
```

Listing 3: Full suite with coverage, then the AI smoke tests

The whole suite runs offline. The AI smoke tests use a fake LLM and a fake web search provider, so no key and no network are needed; storage tests run against an in-memory SQLite database. The one environment requirement is that `DATABASE_URL` points at SQLite in `.env` before the CLI tests run, for the same reason described in the storage section.

Suite	Coverage
src/ (our layer)	87 %
Provided ai/ smoke tests	16/16 passing

7 Deployment & Reproducibility

7.1 Docker

```
docker compose up --build # PostgreSQL + app (API and web page)
docker compose --profile demo up demo # offline demo, no keys, no network
```

Listing 4: Full stack, then the keyless offline demo

`docker compose up -build` starts PostgreSQL and the application and serves the API together with the web page on `http://localhost:8000`; Swagger documentation is at `/docs`. The demo profile is the reproducibility story for graders without keys: it runs the pipeline end to end against canned data.

7.2 API surface

Method	Path	What it does
GET	/health	health check
POST	/api/search	search, used by the web page
POST	/ask	search with an explicit sources choice

```
curl -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"query": "transformer models", "sources": "wikipedia,arxiv"}'
```

Listing 5: Example request

7.3 Configuration

Variable	Needed?	Default	What it does
LLM_PROVIDER	yes	gemini	anthropic, openai or gemini
LLM_MODEL	yes	gemini-2.5-flash	model name
GOOGLE_API_KEY	/ one, for live	—	key for the chosen LLM; not
OPENAI_API_KEY	/ runs	—	needed for the offline demo
ANTHROPIC_API_KEY			
WEB_SEARCH_PROVIDER	yes	tavily	tavily, serper or
TAVILY_API_KEY	/ only for that	—	duckduckgo
SERPER_API_KEY	provider	—	DuckDuckGo needs no key
LOG_LEVEL	no	INFO	DEBUG, INFO, WARNING, ERROR
CACHE_DIR	no	./.cache	where the file cache lives
CACHE_TTL_SECONDS	no	86400	how long a cached answer lasts
PER_SOURCE_TIMEOUT_SECONDS	no	10	timeout per source
DATABASE_URL	no	Postgres container URL	history database; use SQLite locally

The same list ships as `.env.example`; the real `.env` stays local and is gitignored.

8 Limitations & Future Work

- The answer carries [N] markers, but the reference list (title and link) is not yet returned by the API or the CLI. This is the first thing we would finish with another week, since citations without targets are half a feature.

- No authentication and no rate limiting on the API, and every request costs an LLM call — acceptable for a demo, not for anything exposed.
- CORS is open to all origins for the same reason.
- Two caches, the JSON file cache and the database table, do not share keys, so a hit in one is not a hit in the other.
- Tables are created with `create_all`, so there is no migration tool such as Alembic; a schema change means recreating the database.
- The benchmark table uses simulated baseline values from local environment execution.

9 Tools & Acknowledgements

The `ai/` package was provided by the course and is used as distributed. AI coding assistance was used the way any productivity tooling is used — for drafting, boilerplate and first passes at tests — and the time saved went into design, debugging against real provider responses, and verification. Everything in the repository was read, run and tested by us.

References

- [1] Google Gemini API documentation, <https://ai.google.dev/docs>
- [2] Tavily search API documentation, <https://docs.tavily.com>
- [3] Wikipedia Action API (opensearch), <https://www.mediawiki.org/wiki/API:Opensearch>
- [4] arXiv API user manual, <https://info.arxiv.org/help/api/user-manual.html>
- [5] FastAPI documentation, <https://fastapi.tiangolo.com/>
- [6] httpx async HTTP client, <https://www.python-httpx.org/>
- [7] SQLAlchemy documentation, <https://docs.sqlalchemy.org/>

A Appendix — Reproducing the project

```
git clone https://github.com/Eldar2107/research-assistant.git
cd research-assistant
python -m venv venv && source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env # then add your keys

# Offline demo (no keys, no network):
docker compose --profile demo up demo

# Full test suite with coverage:
docker compose run --rm -e PYTHONPATH=/app app pytest --cov=src --cov-report=term-missing

# AI smoke tests:
docker compose run --rm -e PYTHONPATH=/app app pytest tests/test_ai_smoke.py -v

# A single CLI question:
docker compose run --rm -e PYTHONPATH=/app app python -m src.cli "photosynthesis"

# Two sources only, cache bypassed:
```

```
python -m src.cli "How does CRISPR-Cas9 work?" --sources wiki,arxiv --no-cache  
  
# API and web page outside Docker:  
cd frontend && npm install && npm run build && cd ..  
uvicorn src.api:app --reload --port 8000
```

Listing 6: From clone to a running answer